# Product Requirements Document (PRD)

**Product:** Universal Production Error → Engineering Automation Platform[cite: 1]
**Version:** 1.0.0 (MVP to Multi-Phase Implementation)[cite: 1]
**Stack:** .NET Core, PostgreSQL, Redis, Distributed Message Broker[cite: 1]

> **Core Architecture Positioning:** This platform acts as an automated triage, enrichment, and ownership layer between telemetry sources (e.g., Sentry, OpenTelemetry) and issue trackers (e.g., Jira, Azure DevOps)[cite: 1].

## 1. Executive Summary & Pain Points

- **Repeated Exceptions:** Fingerprinting and grouping eliminate noise by aggregating identical occurrences under one incident[cite: 1].
- **Unknown Ownership:** Deterministic routing engine maps issues via code paths, CODEOWNERS, and team hierarchies[cite: 1].
- **Manual Dispatch:** Enriched, canonical work items are created directly in the team's chosen tracker[cite: 1].
- **Security Leakage:** Pre-persistence redaction pipeline scrubs credentials, tokens, and PII from stack traces and context payloads[cite: 1].

## 2. Product Scope Boundaries

| Phase | In-Scope (P0 / Core Essentials) | Out-of-Scope (Explicit Non-Goals) |
|---|---|---|
| **MVP**[cite: 1] | • Multi-tenant registration (Org, App, Service, Env)[cite: 1].<br>• Ingestion API with credential validation & rate limits[cite: 1].<br>• PII/Secret redaction pipeline[cite: 1].<br>• SHA-256 error grouping & fingerprinting[cite: 1].<br>• Hierarchical ownership matcher[cite: 1].<br>• Provider abstraction (`ITicketingProvider`) with 2 adapters[cite: 1].<br>• Deduplicated ticket creation & occurrence counting[cite: 1]. | • Full APM/Observability UI replacement[cite: 1].<br>• Supporting >2 ticket systems day one[cite: 1].<br>• Deep generative AI root-cause analysis[cite: 1].<br>• Irreversible developer auto-assignment[cite: 1]. |
| **V2**[cite: 1] | • Message broker queues (Kafka / RabbitMQ / Azure Service Bus)[cite: 1].<br>• Two-way status synchronization via webhooks[cite: 1].<br>• Linear, GitHub Issues, and ServiceNow adapters[cite: 1].<br>• Commit and deployment correlation engine[cite: 1]. | • Automated code patching / pull request authoring.<br>• Permanent raw payload storage without retention rules[cite: 1]. |

## 3. Functional Requirements

- **FR-01: Multi-Tenancy:** Partition all resources across organizations, applications, and services[cite: 1].
- **FR-02: Secure Ingestion:** Authenticate telemetry payloads over HTTP POST; return `202 Accepted` within <50ms[cite: 1].
- **FR-03: Sanitization:** Automatically scrub sensitive headers (e.g., `Authorization`, `Cookie`) and password fields[cite: 1].

- **FR-04: Deterministic Fingerprinting:** Calculate SHA-256 hash across `exceptionType + normalizedStackFrames + serviceName + routeTemplate` (excluding timestamps/GUIDs)[cite: 1].
- **FR-05: Grouping & Idempotency:** Combine identical events into an `ErrorGroup`; enforce uniqueness using `(organization_id, source_event_id)`[cite: 1].
- **FR-06: Ownership Fallback Hierarchy:** Evaluate path rules → CODEOWNERS → service mapping → module mapping → recent author → team default → team queue → manager escalation[cite: 1].
- **FR-07: Adapter Routing:** Dispatch via the most-specific binding: Environment → Service → App → Team → Org default[cite: 1].
- **FR-08: Retry & Dead-Lettering:** Retry transient failures (immediate, 10s, 30s, 2m, 10m) before routing to a DLQ[cite: 1].

## 4. Core Schema Specifications

```
-- Error Group & Occurrences Schema
CREATE TABLE error_groups (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE,
    fingerprint CHAR(64) NOT NULL,
    exception_type VARCHAR(255) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'Open',
    occurrence_count INT NOT NULL DEFAULT 1,
    first_seen TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_seen TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_org_fingerprint UNIQUE(organization_id, fingerprint)
);

CREATE TABLE error_occurrences (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    error_group_id UUID NOT NULL REFERENCES error_groups(id) ON DELETE CASCADE,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    stack_trace TEXT NOT NULL,
    request_context JSONB,
    version VARCHAR(100),
    correlation_id VARCHAR(255)
);

CREATE TABLE ticket_links (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    error_group_id UUID NOT NULL REFERENCES error_groups(id) ON DELETE CASCADE,
    integration_id UUID NOT NULL REFERENCES ticketing_integrations(id) ON DELETE RESTRICT,
    external_ticket_id VARCHAR(255) NOT NULL,
    status VARCHAR(100) NOT NULL,
    last_synced_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_error_group_integration UNIQUE(error_group_id, integration_id)
);
```

## 5. Provider Adapter Interface Contract

```
public interface ITicketingProvider
{
    Task<TicketResult> CreateAsync(TicketRequest request, CancellationToken ct);
    Task UpdateAsync(string externalTicketId, TicketUpdate request, CancellationToken ct);
    Task AssignAsync(string externalTicketId, string externalUserId, CancellationToken ct);
    Task UpdateStatusAsync(string externalTicketId, TicketStatus status, CancellationToken ct);
    Task<TicketDetails> GetAsync(string externalTicketId, CancellationToken ct);
}
```

## 6. Implementation Roadmap

| Timeline | Deliverables |
| --- | --- |
| **Weeks 1–2**[cite: 1] | Database domain models, EF Core/PostgreSQL setup, Ingestion API scaffolding[cite: 1]. |
| **Weeks 3–4**[cite: 1] | Payload redaction pipeline, SHA-256 fingerprinting, deduplication logic, Error dashboard[cite: 1]. |
| **Weeks 5–6**[cite: 1] | Ownership engine rules, `ITicketingProvider` abstraction, primary adapter (Jira)[cite: 1]. |
| **Weeks 7–8**[cite: 1] | Second adapter (Azure DevOps), message broker decoupling, exponential retry policy, DLQ[cite: 1]. |
| **Weeks 9–10**[cite: 1] | Webhook status sync workers, deployment/commit correlation, alert notifications[cite: 1]. |
| **Weeks 11–12**[cite: 1] | Integration test coverage (xUnit + Testcontainers), Docker/K8s manifests, CI/CD pipelines[cite: 1]. |